EXP NO: 12 End to End communication at transport layer

a) Implement echo client - server using TCP / UDP socket

Aim: To implement an echo client - server program using TCP sockets.

Algorithm;
*) create a server socket
*) Bind the socket to an IP address and port.
*) Listen for client connection.
*) Accept a client connection
*) Receive a message from the client
*) Display the echoed message.

code;

```
import socket
server = socket * socket (socket . AF_TUET, Socket .
                                    sock_stream)
server . bind (( ["127.0.0.1", 5000))
server . Listen (1)
print ("Server waiting for client ...")
conn, addr = server . accept()
print ("connected :", addr)
while True;
        data = conn . recV(1024)
        if not data:
            break
        conn . sendall (data)
    conn . close)
    server . close()
```

Result; the echo client - Server program is implemented successfully using TCP socket.

B) Implement a chat program using socket - programming

Aim: To implement a simple client - server chat program using TCP socket programming.

Code:

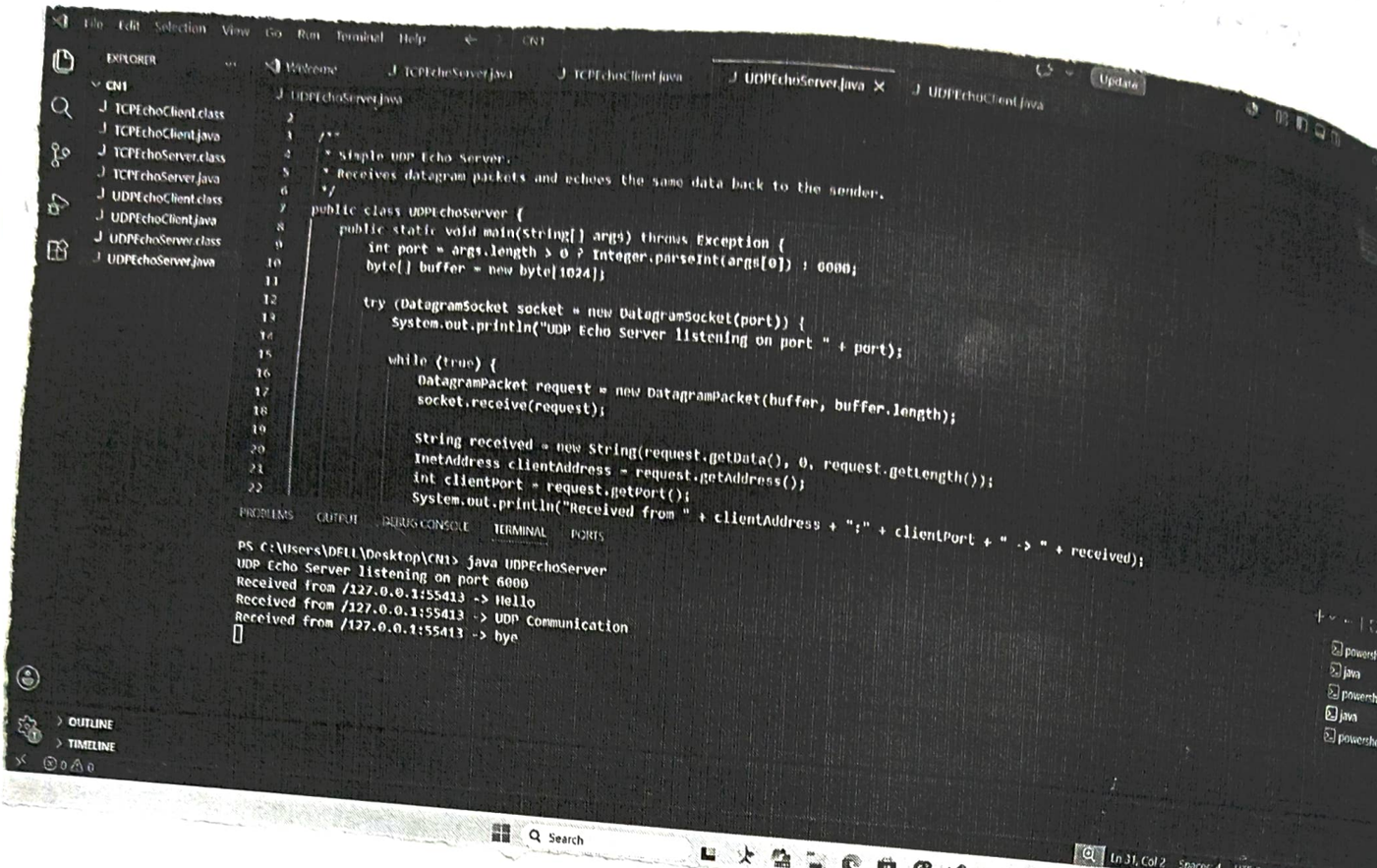
```
import socket
server = socket.socket()
server.bind (("127.0.0.1", 6000))
server.listen(1)
print ("waiting for client...")
conn, addr = server.accept()
print ("client connected")

while true:
    msg = conn.recv(1024).decode()
    if msg.lower() = "bye":
        break
    print ("client:", msg)
    reply = input ("server:")
    conn = sendall (reply.encode())
    if reply.lower() == "bye":
        break
conn.close()
server.close()
```

Procedure:

+) Run the server program first
+) Run the client program in another terminal
+) Exchange message between the client and server.
+) type bye to end the chat

Result: A client - server client program implemented successfully



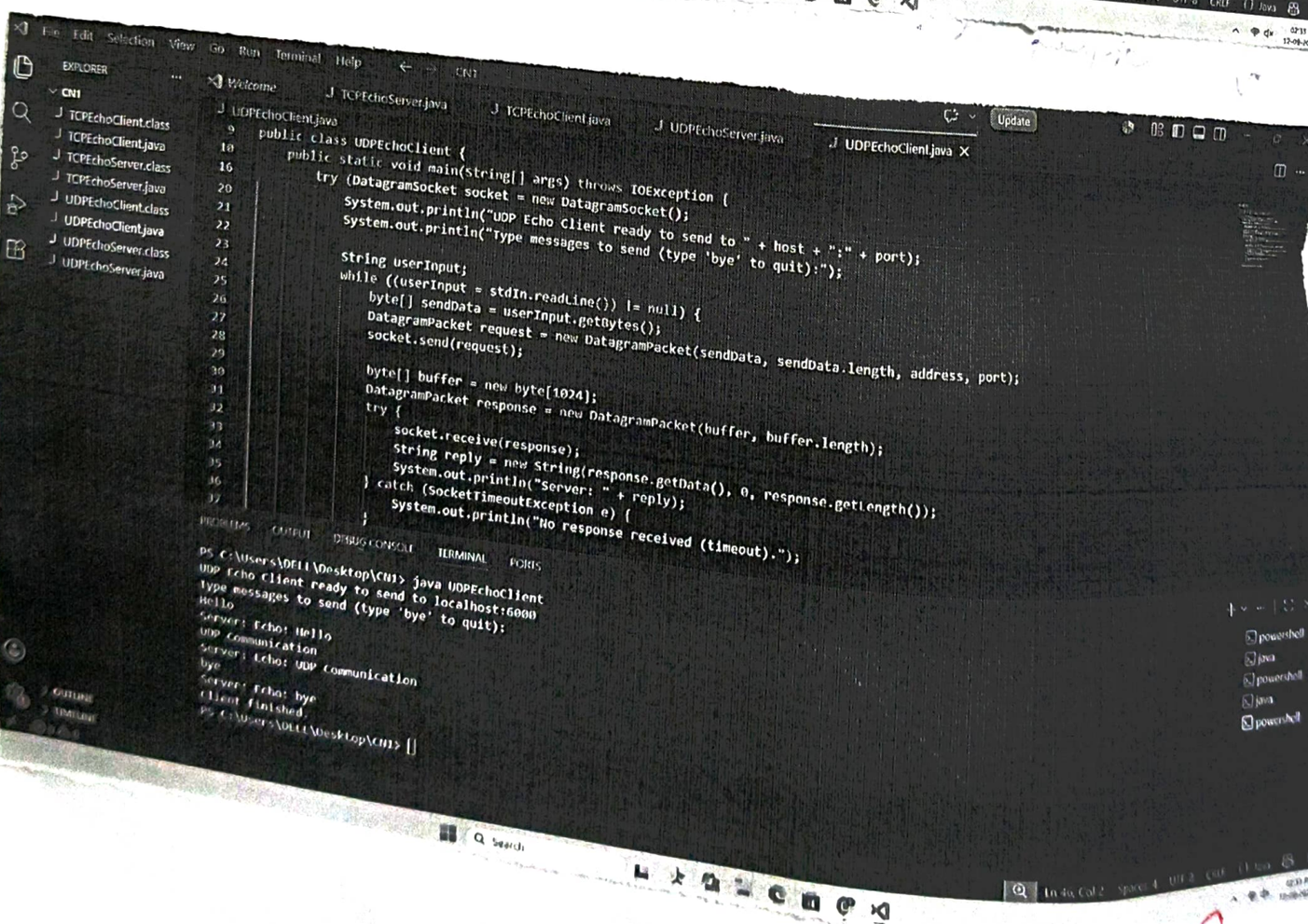
```
/**
 * Simple UDP Echo Server.
 * Receives datagram packets and echoes the same data back to the sender.
 */
public class UDPEchoServer {
    public static void main(String[] args) throws Exception {
        int port = args.length > 0 ? Integer.parseInt(args[0]) : 6000;
        byte[] buffer = new byte[1024];

        try (DatagramSocket socket = new DatagramSocket(port)) {
            System.out.println("UDP Echo Server listening on port " + port);

            while (true) {
                DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                socket.receive(request);

                String received = new String(request.getData(), 0, request.getLength());
                InetAddress clientAddress = request.getAddress();
                int clientPort = request.getPort();
                System.out.println("Received from " + clientAddress + ":" + clientPort + " -> " + received);
```

```
PS C:\Users\DELL\Desktop\CN1> java UDPEchoServer
UDP Echo Server listening on port 6000
Received from /127.0.0.1:55413 -> Hello
Received from /127.0.0.1:55413 -> UDP Communication
Received from /127.0.0.1:55413 -> bye
```



```
public class UDPEchoClient {
    public static void main(String[] args) throws IOException {
        try (DatagramSocket socket = new DatagramSocket();
            System.out.println("UDP Echo Client ready to send to " + host + ":" + port);
            System.out.println("Type messages to send (type 'bye' to quit):");

            String userInput;
            while ((userInput = stdIn.readLine()) != null) {
                byte[] sendData = userInput.getBytes();
                DatagramPacket request = new DatagramPacket(sendData, sendData.length, address, port);
                socket.send(request);

                byte[] buffer = new byte[1024];
                DatagramPacket response = new DatagramPacket(buffer, buffer.length);
                try {
                    socket.receive(response);
                    String reply = new String(response.getData(), 0, response.getLength());
                    System.out.println("Server: " + reply);
                } catch (SocketTimeoutException e) {
                    System.out.println("No response received (timeout).");
                }
```

```
PS C:\Users\DELL\Desktop\CN1> java UDPEchoClient
UDP Echo Client ready to send to localhost:6000
Type messages to send (type 'bye' to quit):
Hello
Server: Echo: Hello
UDP Communication
Server: Echo: UDP Communication
bye
Server: Echo: bye
Client finished.
```